
ShopVerse

Technology Architecture

Deep Dive

Internals, Architecture & Critical Concepts

for every technology in the ShopVerse stack

RabbitMQ	Kafka	PostgreSQL	Redis	MongoDB
Cassandra	Elasticsearch	Neo4j	Prometheus	Docker

Version 1.0 • June 2026 • For Internal Developer Use

ShopVerse — Technology Architecture Deep Dive

Companion document to: [ShopVerse Complete Stack Startup & Data Access Guide v1.1](#)

Purpose: Internals, architecture, and critical concepts for every technology in the ShopVerse stack.

This document does NOT repeat startup commands or connection strings — see the Startup Guide for those.

Version: 1.0 · June 2026 · For internal developer use

Table of Contents

1. [RabbitMQ](#1-rabbitmq)
 2. [Kafka](#2-kafka)
 3. [PostgreSQL](#3-postgresql)
 4. [Redis](#4-redis)
 5. [MongoDB](#5-mongodb)
 6. [Cassandra](#6-cassandra)
 7. [Elasticsearch](#7-elasticsearch)
 8. [Neo4j](#8-neo4j)
 9. [Prometheus & Grafana](#9-prometheus--grafana)
 10. [Docker & Networking](#10-docker--networking)
 11. [Cross-Technology Order Placement Flow](#11-cross-technology-order-placement-flow)
-

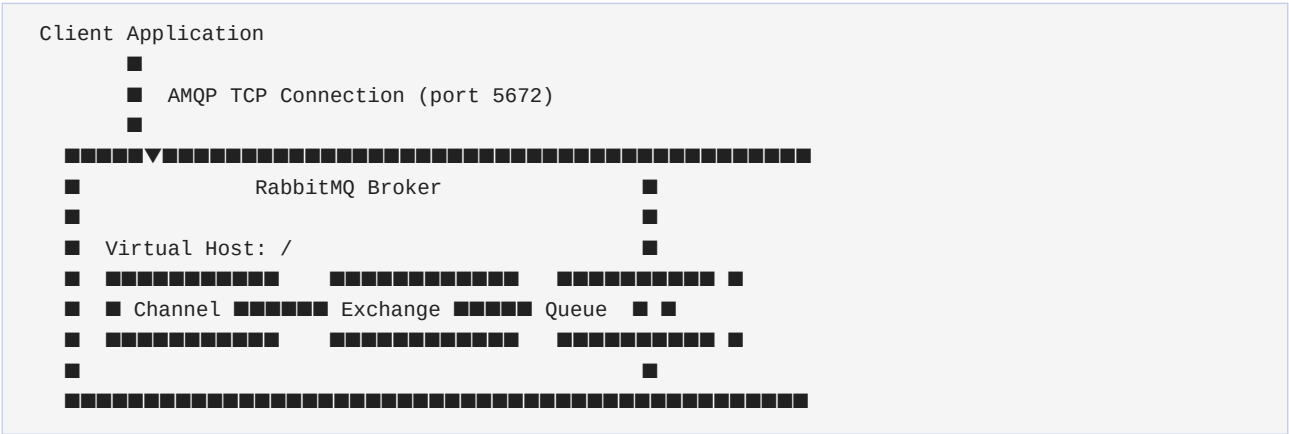
1. RabbitMQ

1.1 What It Is & Why ShopVerse Uses It

RabbitMQ is a message broker implementing the AMQP 0-9-1 protocol. Unlike Kafka (which is a distributed log), RabbitMQ is a traditional push-based queue broker: messages are routed through exchanges to queues and consumed by workers, then deleted after acknowledgement. ShopVerse uses RabbitMQ for **transient, consumer-driven messaging** — specifically email/SMS notifications, payment gateway callbacks, and merchant webhook delivery. These are fire-and-forget tasks where message durability after consumption is irrelevant but routing flexibility and per-message TTL matter greatly.

1.2 AMQP Protocol Overview

AMQP (Advanced Message Queuing Protocol) defines a binary wire protocol for message brokers.



Key protocol layers:

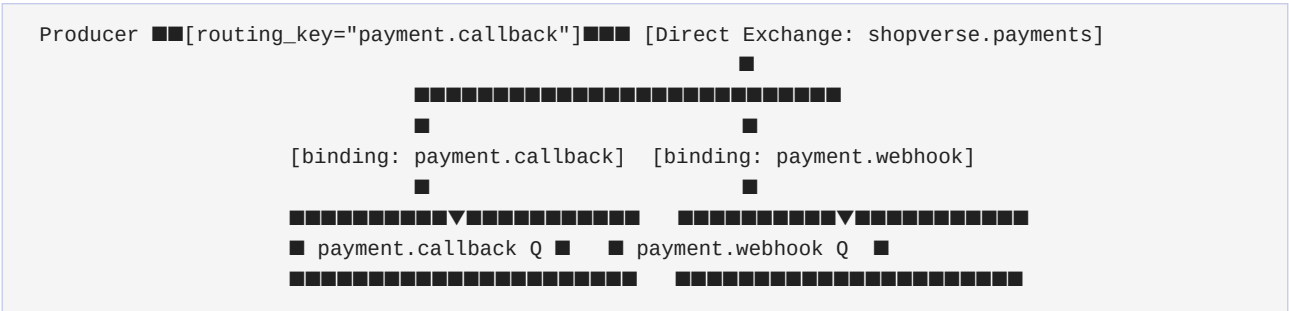
Layer	Description
Connection	A single TCP connection between client and broker. Expensive to create — reused via connection pooling.
Channel	A lightweight virtual connection multiplexed over one TCP connection. Each thread should use its own channel.
Virtual Host	An isolated namespace (like a database schema) with its own exchanges, queues, and permissions. ShopVerse uses <code>/</code> (the default vhost).
Exchange	Receives messages from producers and routes them to queues based on routing rules.
Queue	A buffer that stores messages until a consumer picks them up.
Binding	A rule linking an exchange to a queue, optionally with a routing key pattern.

1.3 Exchange Types

RabbitMQ has four exchange types. ShopVerse uses three of them.

Direct Exchange

Routes messages to queues where the binding key **exactly matches** the routing key.



Topic Exchange

Routes messages to queues where the binding key matches a **pattern** using ***** (one word) and **#** (zero or more words).

Producer ■■[routing_key="notification.email.order.confirmed"]■■■ [Topic Exchange: shopverse.notifications]

[binding: notification.email.#] [binding: notification.sms.#]

■ queue.email ■

■ queue.sms ■

notification.email.order.confirmed → matches notification.email.# → goes to queue.email ✓
notification.sms.order.confirmed → matches notification.sms.# → goes to queue.sms ✓
notification.push.order.confirmed → matches neither → dropped ✗

Fanout Exchange

Broadcasts messages to **all bound queues** regardless of routing key.

Producer ■■[any routing key]■■■ [Fanout Exchange: shopverse.webhooks]

■ (future queues)

■ webhook.delivery Q ■

Every message goes to every bound queue – routing key is ignored.

Headers Exchange (not used in ShopVerse)

Routes based on message header attributes rather than routing key. Rarely used; included for completeness.

1.4 Queue Properties

Property	Description	ShopVerse Usage
durable	Queue survives broker restart	All ShopVerse queues are durable
exclusive	Queue deleted when connection closes	Not used
auto-delete	Queue deleted when last consumer disconnects	Not used
x-message-ttl	Per-queue message TTL in ms — expired messages go to DLX	email/sms: 300000ms (5 min), payment.callback: 60000ms (1 min), webhook: 600000ms (10 min)
x-dead-letter-exchange	Exchange to route expired/rejected messages	`shopverse.dlx` on all queues except payment.webhook
x-dead-letter-routing-key	Routing key used when sending to DLX	`email.dlx` for email queue

Exchange Name	Type	Durable	Purpose
`shopverse.webhooks`	Fanout	Yes	Broadcasts order events to all registered merchant webhook endpoints
`shopverse.dlx`	Direct	Yes	Dead Letter Exchange — receives expired/rejected messages from all queues

Queues

Queue Name	Bound Exchange	Routing Key	TTL	DLX	DLQ	Purpose
`shopverse.queue.email`	`shopverse.notifications`	`notification.email.#`	5 min	`shopverse.dlx`	`shopverse.queue.email.dlq`	Email delivery for order/payment events
`shopverse.queue.sms`	`shopverse.notifications`	`notification.sms.#`	5 min	`shopverse.dlx`	—	SMS delivery (future use)
`shopverse.queue.email.dlq`	`shopverse.dlx`	`email.dlq`	None	None	—	Permanently failed email messages
`shopverse.queue.payment.callback`	`shopverse.payments`	`payment.callback`	1 min	`shopverse.dlx`	—	Async payment gateway callback processing
`shopverse.queue.payment.webhook`	`shopverse.payments`	`payment.webhook`	None	None	—	Payment webhook idempotency verification
`shopverse.queue.webhook.delivery`	`shopverse.webhooks`	(fanout, ignored)	10 min	`shopverse.dlx`	—	Merchant webhook HTTP delivery

Bindings

Exchange	Routing Key	Queue	Match Type
`shopverse.notifications`	`notification.email.#`	`shopverse.queue.email`	Pattern — matches any routing key starting with `notification.email.`
`shopverse.notifications`	`notification.sms.#`	`shopverse.queue.sms`	Pattern — matches any routing key starting with `notification.sms.`
`shopverse.dlx`	`email.dlq`	`shopverse.queue.email.dlq`	Exact match
`shopverse.payments`	`payment.callback`	`shopverse.queue.payment.callback`	Exact match

Exchange	Routing Key	Queue	Match Type
`shopverse.payments`	`payment.webhook`	`shopverse.queue.payment.webhook`	Exact match
`shopverse.webhooks`	(any)	`shopverse.queue.webhook.delivery`	Fanout — routing key ignored

Routing Keys Used by Producer

Event Type	Producer Class	Exchange	Routing Key
OrderConfirmationNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.order.confirmed`
OrderShippedNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.order.shipped`
OrderDeliveredNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.order.delivered`
OrderCancelledNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.order.cancelled`
PaymentSuccessNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.payment.success`
PaymentFailedNotification	`NotificationRabbitPublisher`	`shopverse.notifications`	`notification.email.payment.failed`
Payment callback (raw)	`NotificationRabbitPublisher`	`shopverse.payments`	`payment.callback`
Webhook event (raw)	`NotificationRabbitPublisher`	`shopverse.webhooks`	`""` (fanout)

Consumers

Consumer Class	Listens On	ACK Mode	Concurrency	Action
`EmailNotificationConsumer.handleEmailNotification()`	`shopverse.queue.email`	Manual (exception = nack → DLQ)	3–10	Resolves subject/body from NotificationEvent, logs simulated email send
`EmailNotificationConsumer.handleDlq()`	`shopverse.queue.email.dlq`	Auto	3–10	Logs DLQ receipt, alerts on-call in production
`PaymentCallbackConsumer.handlePaymentCallback()`	`shopverse.queue.payment.callback`	Manual	3–10	Parses SUCCESS/FAILED status, publishes PaymentSuccess/FailedNotification back to RabbitMQ
`PaymentCallbackConsumer.handlePaymentWebhook()`	`shopverse.queue.payment.webhook`	Manual	3–10	Verifies webhook ID for idempotency

Consumer Class	Listens On	ACK Mode	Concurrency	Action
`WebhookDeliveryConsumer.handleWebhookDelivery()`	`shopverse.queue.webhook.delivery`	Manual	3–10	HTTP POSTs to merchant's registered webhook URL with HMAC-SHA256 signature

1.7 End-to-End Notification Flow

```

[Order placed by customer]
  ■
  ▼
OrderKafkaConsumer.handleOrderPlaced()
  ■
  ■ calls NotificationPublisher.publish(
  ■     OrderConfirmationNotification)
  ▼
NotificationRabbitPublisher.publish()
  ■ resolveRoutingKey() → "notification.email.order.confirmed"
  ■ rabbitTemplate.convertAndSend(
  ■     "shopverse.notifications",
  ■     "notification.email.order.confirmed",
  ■     event)
  ▼
[Exchange: shopverse.notifications] (Topic)
  ■ routing key matches "notification.email.#"
  ▼
[Queue: shopverse.queue.email]
  ■
  ▼
EmailNotificationConsumer.handleEmailNotification()
  ■ resolveSubject() → "Order #X Confirmed – ShopVerse"
  ■ resolveBody()    → "Your order #X totalling $Y confirmed"
  ■ sendEmail()      → logs / in production: calls SendGrid/SES
  ■
  SUCCESS → ACK → message deleted
  FAILURE → throws RuntimeException
            → NACK (defaultRequeueRejected=false)
            → [DLX: shopverse.dlx]
            → [Queue: shopverse.queue.email.dlq]
            → EmailNotificationConsumer.handleDlq()

→ logs + alerts

```

1.8 Publisher Confirms

Publisher confirms ensure messages reach the broker (not just the TCP buffer).


```

Producer → broker.basicPublish()
    ■
    Broker receives and persists to disk
    ■
    Broker sends back ConfirmCallback(ack=true)
    ■
    If ack=false → log/alert "RabbitMQ NACK"

```

Configured in `RabbitMQConfig.rabbitTemplate()` via `setConfirmCallback`.

1.9 Consumer Concurrency

`SimpleRabbitListenerContainerFactory` is configured with:

- `concurrentConsumers` = 3 — starts with 3 consumer threads per listener
- `maxConcurrentConsumers` = 10 — scales up to 10 under load
- `defaultRequeueRejected` = false — on exception, dead-letter instead of re-queue loop

1.10 Monitoring Signals

Signal	Metric / Where to Check	Alert Threshold
Queue depth growing	RabbitMQ UI → Queues tab → Messages	> 1000 messages ready
DLQ has messages	`shopverse.queue.email.dlq` message count	Any > 0
Consumer count drops	Queues tab → Consumers column	< 1 consumer on email queue
Message publish rate	Overview tab → Publish rate	Sudden drop to 0 during order activity
Channel count	Connections tab	> 50 channels per connection

■ Things That Will Break in Production

- Not setting `defaultRequeueRejected=false` — a failing consumer will loop forever, consuming CPU and never DLQ-ing the message.
- Forgetting ``durable=true`` on queues/exchanges — broker restart loses all queued messages.
- Using the same channel across threads — channels are not thread-safe; concurrent sends on one channel corrupt frames.
- Missing DLQ consumer — messages pile up in the DLQ silently; you'll never know notifications failed.
- TTL on `payment.callback`` queue set too low (1 min) — if the Spring Boot app is slow to start or restarting, payment callbacks expire before processing, causing silent payment reconciliation failures.

2. Kafka

2.1 What It Is & Why ShopVerse Uses It

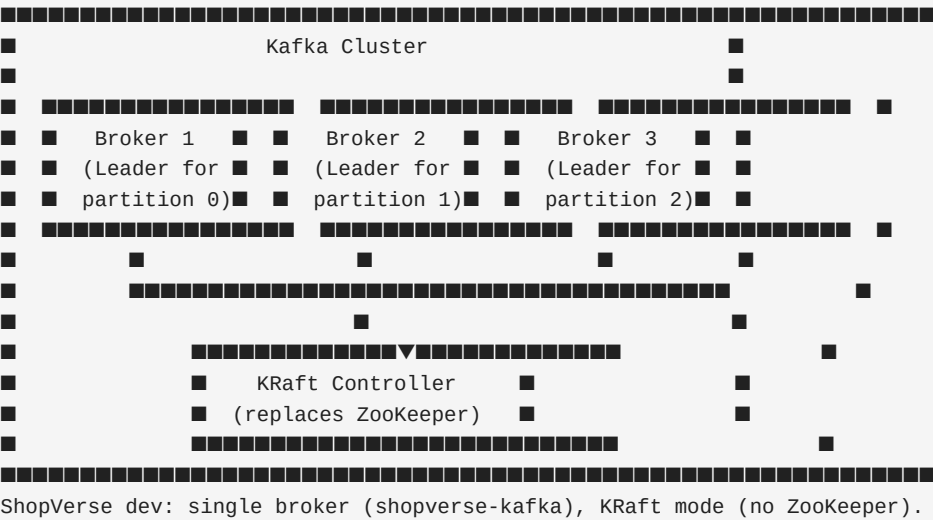
Apache Kafka is a distributed, append-only event log. Unlike RabbitMQ, messages are retained after consumption and consumers track their own position (offset). ShopVerse uses Kafka for **durable, replayable, ordered event streaming** — order lifecycle events, inventory changes, analytics, and product sync. Kafka's consumer group model lets multiple independent services (inventory, notifications, search sync) each consume the same event independently without coupling.

2.2 Kafka vs Traditional Message Queue

```
RabbitMQ (Push-based Queue):
  Producer → [Queue] → Consumer → ACK → Message DELETED
  - Broker pushes messages to consumer
  - Message gone after consumption
  - Multiple consumers compete for messages (one wins)

Kafka (Pull-based Log):
  Producer → [Topic Partition: offset 0,1,2,3...] → Consumer pulls at own pace
  - Consumer pulls messages at its own offset
  - Message retained for configured retention period
  - Multiple consumer groups each get ALL messages independently
```

2.3 Cluster Architecture



2.4 Topic Anatomy

```
Partition 0: [offset 0][offset 1][offset 2]...[offset N]
Partition 1: [offset 0][offset 1][offset 2]...[offset M]
Partition 2: [offset 0][offset 1][offset 2]...[offset K]
```

- Messages with the same KEY always go to the same partition (orderId → consistent partition)
- Within a partition, order is GUARANTEED
- Across partitions, order is NOT guaranteed
- Replication factor 1 = no replicas (dev only; use 3 in production)

Setting	ShopVerse Value	Effect
acks	Default (`1`)	Leader confirms write; followers may not have it yet
batch.size	Default (16KB)	Messages batched before sending for throughput
linger.ms	Default (`0`)	No artificial delay — send immediately
idempotent producer	Not configured	Risk of duplicate messages on retry
key	`orderId.toString()`	Ensures all events for one order go to same partition (ordering guarantee)

[illegible]

Consumer Group: shopverse-dlt (DLT listener) – separate group, reads DLT topic

Before rebalance:

T1 → P0, P1

T2 → P2

New consumer T3 joins:

[REBALANCE: all consumers stop]

T1 → P0

T2 → P1

T3 → P2

[REBALANCE complete: all consumers resume]

During rebalance: NO messages are consumed. This causes latency spikes.

2.7 Delivery Semantics

Semantic	How	ShopVerse Risk
At-most-once	Commit offset before processing	Message lost if app crashes mid-process
At-least-once	Commit offset after processing (ShopVerse default)	Duplicate messages if app crashes after process but before commit
Exactly-once	Transactional producer + idempotent consumer	Not configured in ShopVerse

ShopVerse uses at-least-once. Consumers should be idempotent (handling the same event twice safely).

2.8 Retry Topics & Dead Letter Topics (@RetryableTopic)

Normal flow:

shopverse.orders → OrderKafkaConsumer → SUCCESS → commit offset

Failure flow:

shopverse.orders → OrderKafkaConsumer → EXCEPTION

■

▼ (attempt 1 failed)

shopverse.orders-retry-1000 (wait 1s)

■

▼ (attempt 2 failed)

shopverse.orders-retry-2000 (wait 2s)

■

▼ (attempt 3 failed – all retries exhausted)

shopverse.orders.DLT → consumeDlt() → logs + alert

2.9 ShopVerse Kafka Topic Inventory

shopverse.orders

Field	Value
Partitions	3
Replicas	1
Key	`orderId` (String)
Consumer group	`shopverse-consumer`
Retry topics	`shopverse.orders-retry-1000`, `shopverse.orders-retry-2000`
DLT	`shopverse.orders.DLT` (consumed by `shopverse-dlt` group)
Producer class	`OrderKafkaProducer`
Consumer class	`OrderKafkaConsumer`

Events & Payload Fields:

Event	Fields	Trigger
`OrderPlaced`	`orderId`, `customerId`, `total`, `occurredAt`	Customer places order via `POST /api/orders`
`OrderConfirmed`	`orderId`, `occurredAt`	Admin confirms via `PATCH /api/orders/{id}/confirm`
`OrderShipped`	`orderId`, `trackingNumber`, `occurredAt`	Admin ships via `PATCH /api/orders/{id}/ship`
`OrderDelivered`	`orderId`, `occurredAt`	Admin marks delivered via `PATCH /api/orders/{id}/deliver`
`OrderCancelled`	`orderId`, `reason`, `occurredAt`	Customer/admin cancels
`OrderRefunded`	`orderId`, `refundAmount`, `occurredAt`	Admin refunds

■■ **Known issue:** No `eventType` field in the JSON payload. Consumers must infer the event type from which fields are present, or rely on the `__TypeId__` Kafka message header.

Consumer actions per event:

Event	Action in `OrderKafkaConsumer`
`OrderPlaced`	Publish `OrderConfirmationNotification` to RabbitMQ + publish `StockReserved` to `shopverse.inventory`
`OrderConfirmed`	No-op (confirmation already sent on OrderPlaced)
`OrderShipped`	Publish `OrderShippedNotification` to RabbitMQ
`OrderDelivered`	Publish `OrderDeliveredNotification` to RabbitMQ
`OrderCancelled`	Publish `StockReleased` to `shopverse.inventory` + publish `OrderCancelledNotification` to RabbitMQ
`OrderRefunded`	Publish `StockReleased` to `shopverse.inventory` + publish `PaymentSuccessNotification` (as refund) to RabbitMQ

shopverse.inventory

Field	Value
Partitions	3
Replicas	1
Key	`orderId` (String)
Consumer group	`shopverse-inventory-consumer`
Retry topics	`shopverse.inventory-retry-2000`, `shopverse.inventory-retry-4000`
DLT	`shopverse.inventory.DLT`
Producer class	`OrderKafkaConsumer` (publishes from order event handlers)
Consumer class	`InventoryKafkaConsumer`

Events & Payload Fields:

Event	Fields	Trigger
`StockReserved`	`productId`, `quantity`, `orderId`, `occurredAt`	`OrderPlaced` handling in `OrderKafkaConsumer`
`StockReleased`	`productId`, `quantity`, `orderId`, `reason`, `occurredAt`	`OrderCancelled` or `OrderRefunded` handling
`StockLow`	`productId`, `remainingStock`, `threshold`, `occurredAt`	`InventoryService` threshold check
`StockExhausted`	`productId`, `occurredAt`	Stock hits zero

■■ **Known issue:** `StockReplenished` is defined in `InventoryEvent` sealed interface but **never published anywhere** — `restoreStock()` in `InventoryKafkaConsumer` silently updates the DB without firing this event.

Consumer actions per event:

Event	Action in `InventoryKafkaConsumer`
`StockReserved`	Audit log only (stock already reduced by `PlaceOrderUseCase`)
`StockReleased`	`restoreStock()` — increments `stock_count` in PostgreSQL `products` table
`StockLow`	Warning log; production: trigger reorder workflow / alert procurement
`StockExhausted`	Publishes `ProductEvent.StockDepleted` via Spring `EventPublisher` → `ProductSyncService` marks product out-of-stock in Elasticsearch

shopverse.analytics

Field	Value
Partitions	6 (high-volume)
Replicas	1
Key	`sessionId` (String)
Consumer group	`shopverse-analytics-consumer`
Retry topics	`shopverse.analytics-retry-500`, `shopverse.analytics-retry-1000`
DLT	`shopverse.analytics.DLT`
Producer class	`OrderKafkaProducer` (via `EventPublisher`)
Consumer class	`AnalyticsKafkaConsumer`

Events & Payload Fields:

Event	Fields	Triggered by UI Action
`ProductViewed`	`productId`, `productName`, `category`, `customerId`, `sessionId`, `occurredAt`	Frontend calls `POST /api/analytics/track/view` on product detail page load
`ProductSearched`	`query`, `resultsCount`, `sessionId`, `occurredAt`	Frontend calls `POST /api/analytics/track/search` on search
`ProductAddedToCart`	`productId`, `quantity`, `customerId`, `sessionId`, `occurredAt`	Frontend calls `POST /api/analytics/track/cart` on Add to Cart click
`CheckoutStarted`	`customerId`, `cartTotal`, `itemCount`, `sessionId`, `occurredAt`	Frontend calls `POST /api/analytics/track` with `eventType=CHECKOUT_STARTED`
`OrderConverted`	`orderId`, `customerId`, `total`, `sessionId`, `occurredAt`	Frontend calls `POST /api/analytics/track` with `eventType=ORDER_CONVERTED`

■ ■ **Known issue (fixed):** Frontend was posting to wrong paths (</analytics/view> instead of </analytics/track/view>). Fixed in `client.js`. Backend never auto-fires analytics — entirely frontend-driven.

shopverse.products

Field	Value
Partitions	3
Replicas	1
Key	`productId` (String)
Consumer group	None (Spring `@TransactionalEventListener` — not a Kafka consumer)
DLT	None configured
Producer class	`OrderKafkaProducer` (via `EventPublisher`)

Field	Value
Sync consumer	`ProductSyncService` (Spring event listener, not Kafka consumer)

Events & Payload Fields:

Event	Fields	Trigger
`ProductCreated`	`productId`, `name`, `occurredAt`	`CreateProductUseCase.execute()`
`ProductUpdated` (field="updated")	`productId`, `field`, `occurredAt`	`UpdateProductUseCase.execute()`
`ProductUpdated` (field="deleted")	`productId`, `field="deleted"`, `occurredAt`	`DeleteProductUseCase.execute()`
`StockDepleted`	`productId`, `occurredAt`	`InventoryKafkaConsumer` on `StockExhausted`

■ Known issues:

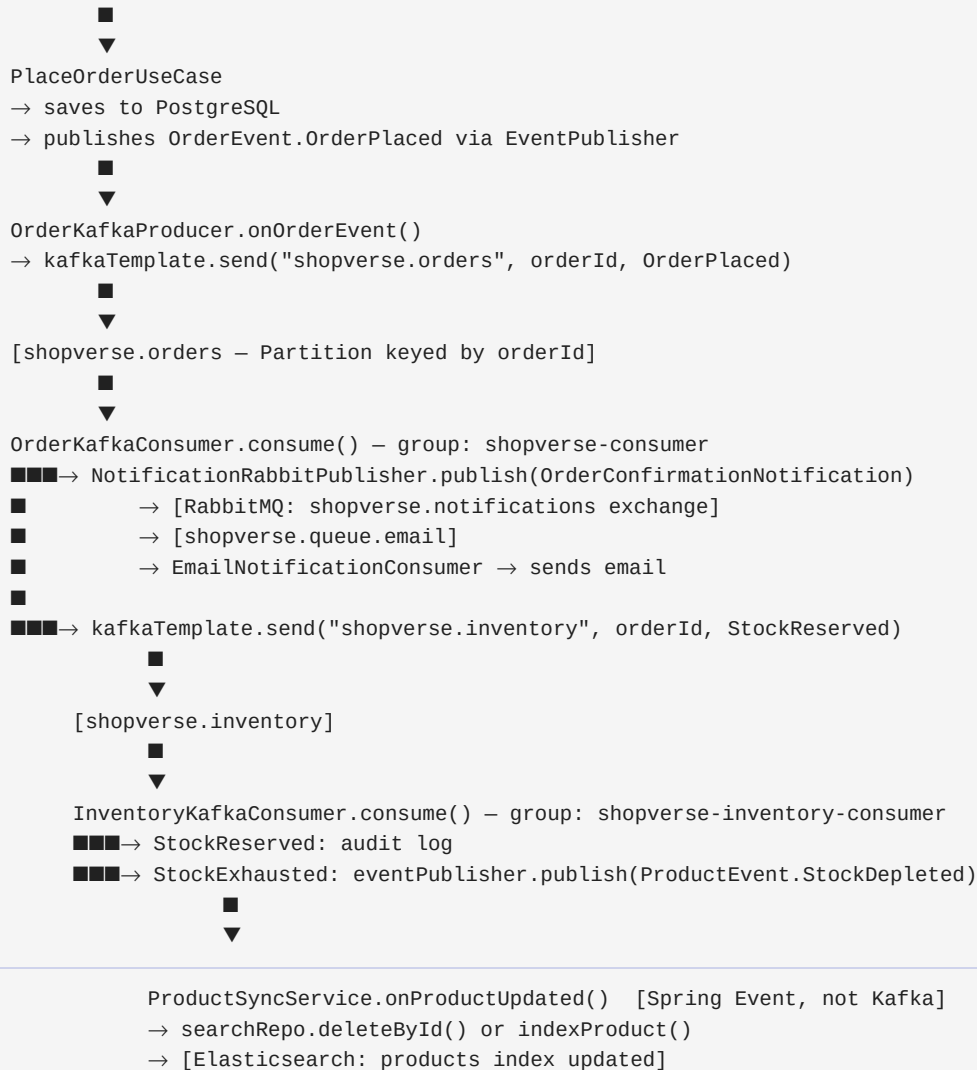
1. Delete is disguised as ProductUpdated with field="deleted" — no dedicated ProductDeleted event.
2. StockReplenished is in the sealed interface but never published.

shopverse.notifications

Field	Value
Partitions	2
Replicas	1
Key	`orderId` (String)
Notes	Topic exists in Kafka but ShopVerse routes notifications via RabbitMQ, not this topic. Reserved for future use.

2.10 Cross-Topic Flow Diagram

[POST /api/orders] – Customer places order



2.11 Monitoring Signals

Signal	How to Check	Alert Threshold
Consumer lag	`kafka-consumer-groups --describe`	Lag > 1000 on any topic
DLT has messages	`kafka-console-consumer --topic shopverse.orders.DLT`	Any message
Broker offline	`kafka-topics --list` fails	Immediate alert
Retry topic growing	Describe `shopverse.orders-retry-*`	Offset advancing rapidly

■ Things That Will Break in Production

- Replication factor of 1 — a single broker failure loses all data. Use RF=3 in production.
- No `eventType` discriminator in payload — consumers relying on field inference will misroute events if a new field is added to an existing event type.
- Consumer group rebalance during high load — all consumption stops during rebalance. Add `session.timeout.ms` and `max.poll.interval.ms` tuning for long-running consumers.
- `StockReserved` sent with `productId=null, quantity=0` in `OrderKafkaConsumer` — the inventory consumer receives an audit-only event with no usable data. Real stock deduction happens in `PlaceOrderUseCase` before the event is sent.
- Analytics topic consumer group not configured — if `AnalyticsKafkaConsumer` fails silently, analytics data is lost with no DLT alert.

3. PostgreSQL

3.1 What It Is & Why ShopVerse Uses It

PostgreSQL 16 is ShopVerse's primary relational database — the source of truth for orders, products, customers, and inventory. It was chosen over MySQL for JSONB support (audit log), native table partitioning, advanced window functions, stored procedures, triggers, and strong MVCC (no lock contention on reads).

3.2 MVCC — Multi-Version Concurrency Control

Every row in PostgreSQL has hidden system columns `xmin` (transaction that created it) and `xmax` (transaction that deleted/updated it). Readers never block writers.

```
Row in orders table: id=1, status='PENDING', xmin=100, xmax=0

T1 (xid=101): UPDATE orders SET status='CONFIRMED' WHERE id=1
→ Creates new row version: status='CONFIRMED', xmin=101, xmax=0
→ Marks old row:          status='PENDING',   xmin=100, xmax=101

T2 (xid=102, REPEATABLE READ snapshot at xid=100):
→ Sees status='PENDING' (xmin=100 ≤ snapshot, xmax=101 > snapshot)
→ T1's changes are INVISIBLE to T2 until T2 starts a new transaction

T3 (xid=103, starts AFTER T1 commits):
→ Sees status='CONFIRMED'
→ Old row version will be cleaned up by VACUUM
```

3.3 WAL — Write-Ahead Log

Every change is written to the WAL before it touches the actual table files, ensuring durability.

```

Client sends UPDATE
    ■
    ▼
PostgreSQL writes to WAL (pg_wal/) ← fast sequential write
    ■
    ▼
Returns SUCCESS to client
    ■
    ▼ (asynchronously)
Applies change to heap (table data files)

On crash: replays WAL to reconstruct committed state
For replication: WAL segments shipped to replica

```

3.4 ShopVerse Table Inventory

Table	Purpose	Primary Key	Notable Columns
`customers`	Customer accounts	`id BIGSERIAL`	`email UNIQUE`, `tier ENUM`, `payment_token` (AES-256 encrypted), `preferred_zone_id`
`products`	Product catalog	`id BIGSERIAL`	`sku UNIQUE`, `stock_count INT`, `version INT` (@Version for optimistic lock), `active BOOLEAN`, `tags TEXT[]`
`orders`	Order records	`id BIGSERIAL`	`customer_id FK`, `status ENUM CHECK`, `total NUMERIC(19,2)`, `created_at TIMESTAMPTZ`
`order_items`	Line items per order	`(order_id, product_id)` composite	`quantity INT`, `unit_price NUMERIC(19,2)`
`order_audit_log`	Trigger-populated audit trail	`id BIGSERIAL`	`table_name`, `operation`, `old_data JSONB`, `new_data JSONB`, `changed_at TIMESTAMPTZ`
`promotions`	Discount promotions (JOINED inheritance)	`id BIGSERIAL`	`type DISCRIMINATOR`, `discount_value NUMERIC`
`es_sync_log`	Elasticsearch sync tracking (Liquibase-managed)	`id BIGSERIAL`	`product_id`, `synced_at`, `status`

3.5 Flyway Migration History

Version	File	What It Does
V1	`V1__initial_schema.sql`	Creates all core tables with constraints, CHECK constraints on `orders.status`

Version	File	What It Does
V2	`V2__add_columns.sql`	Adds `discount_amount NUMERIC` to `orders`
V3	`V3__indexes.sql`	All performance indexes (see below)
V4	`V4__stored_procedure.sql`	Creates `sp_complete_order` stored procedure
V5	`V5__procedures.sql`	Creates `fn_customer_lifetime_value` function
V6	`V6__triggers.sql`	Creates `trg_orders_audit` and `trg_prevent_status_regression` triggers
V7	`V7__partitioning.sql`	Converts `orders` to range-partitioned table; creates monthly partitions
V8	`V8__add_partitioning_and_triggers.sql`	Adds indexes on partitions, partition-aware trigger updates

3.6 Index Inventory (V3__indexes.sql)

Index Name	Table	Type	Columns	Condition	Serves
`idx_orders_customer`	`orders`	B-Tree	`customer_id`	—	`GET /api/orders/customer/{id}`
`idx_orders_pending`	`orders`	Partial B-Tree	`customer_id`	`WHERE status = 'PENDING'`	Pending order lookups without full scan
`idx_orders_cov`	`orders`	Covering B-Tree	`customer_id` INCLUDE `status`, total, created_at`	—	List queries — index-only scan, no heap access
`idx_order_items_comp`	`order_items`	Composite B-Tree	`order_id`, product_id`	—	Join between orders and order_items
`idx_products_tags`	`products`	GIN	`tags`	—	`@> '{tag}'` array contains queries
`idx_products_lower_name`	`products`	Expression B-Tree	`LOWER(name)`	—	Case-insensitive name search
`idx_orders_customer_jpa`	`orders`	B-Tree	`customer_id`	—	JPA `@Table(indexes=...)` declaration
`idx_orders_status_created`	`orders`	B-Tree	`status, created_at`	—	Status + date range queries

3.7 Stored Procedures & Triggers

`sp_complete_order(p_order_id BIGINT)`

Called via `@Procedure(procedureName="sp_complete_order")` in `OrderRepository`.

```
Step 1: UPDATE orders SET status = 'DELIVERED' WHERE id = p_order_id
Step 2: DECREMENT products.stock_count for each item in order_items
Step 3: INSERT INTO order_audit_log (operation='COMPLETE', ...)
All three steps in one atomic transaction – if any fails, all roll back.
```

`fn_customer_lifetime_value(customer_id BIGINT) RETURNS NUMERIC`

Called via `SimpleJdbcCall` in the application.

```
SELECT SUM(total) FROM orders
WHERE customer_id = $1
AND status NOT IN ('CANCELLED', 'REFUNDED')
```

`trg_orders_audit (AFTER INSERT OR UPDATE OR DELETE ON orders)`

```
FOR EACH ROW:
  INSERT INTO order_audit_log(
    table_name = 'orders',
    operation   = TG_OP,          -- 'INSERT', 'UPDATE', 'DELETE'
    old_data    = row_to_json(OLD), -- NULL on INSERT
    new_data    = row_to_json(NEW), -- NULL on DELETE
    changed_at  = NOW()
  )
```

`trg_prevent_status_regression (BEFORE UPDATE ON orders)`

```
Valid transitions: PENDING→CONFIRMED→PROCESSING→SHIPPED→DELIVERED
                  CONFIRMED→CANCELLED
IF new.status goes backwards THEN
  RAISE EXCEPTION 'Invalid status transition: % → %', OLD.status, NEW.status
END IF
```

3.8 Table Partitioning

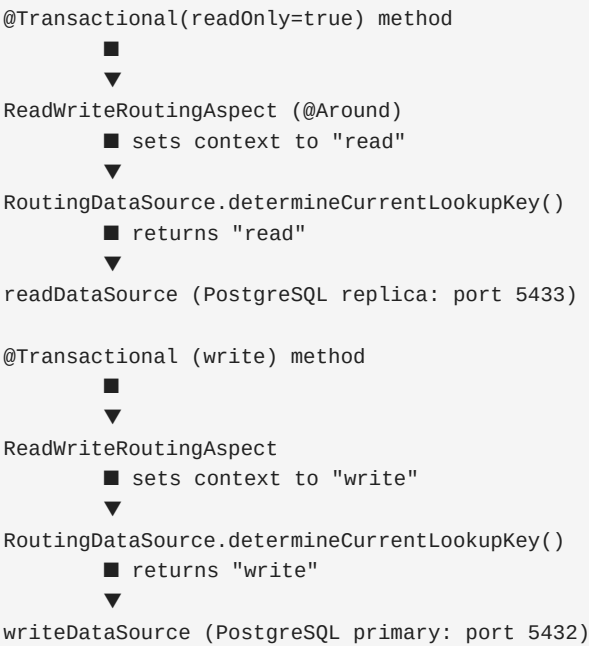
`orders` is partitioned by `RANGE(created_at)` into monthly partitions.

```
orders (parent – no data stored here)
■■■■ orders_2025_01 PARTITION OF orders FOR VALUES FROM ('2025-01-01') TO ('2025-02-01')
■■■■ orders_2025_02 PARTITION OF orders FOR VALUES FROM ('2025-02-01') TO ('2025-03-01')
■■■■ ...
■■■■ orders_2026_06 PARTITION OF orders FOR VALUES FROM ('2026-06-01') TO ('2026-07-01')
■■■■ orders_default PARTITION OF orders DEFAULT ← catches any out-of-range rows
```

`PartitionManagerJob` runs `@Scheduled(cron="0 0 1 *")` — creates next month's partition + its indexes if not yet existing.

Effect on Hibernate: Queries with `WHERE created_at BETWEEN ...` automatically prune irrelevant partitions. Without partition pruning, the query planner would scan all partitions.

3.9 Read/Write Routing



3.10 @Transactional Isolation Levels

Method	Class	Isolation	Reason
`placeOrder()`	`OrderApplicationService`	`REPEATABLE_READ`	Prevents phantom reads on stock count during concurrent order placement
`charge()`	`PaymentService`	`REQUIRES_NEW`	Payment rolls back independently if outer transaction fails
`write()`	`AuditService`	`NOT_SUPPORTED`	Always runs outside caller's transaction — audit must persist even if order rolls back
`generateDailyReport()`	`ReportService`	`SERIALIZABLE`	Consistent snapshot across all tables for accurate aggregation

3.11 Monitoring Signals

Signal	Query	Alert
Slow queries	`pg_stat_statements WHERE mean_exec_time > 100`	Any query p99 > 200ms
Replication lag	`pg_stat_replication` lag column	> 50 MB

Signal	Query	Alert
Dead tuples (VACUUM needed)	<code>`pg_stat_user_tables.n_dead_tup`</code>	> 10% of live tuples
Lock waits	<code>`pg_stat_activity JOIN pg_locks`</code>	Any wait > 30s
Connection pool saturation	HikariCP <code>`hikaricp_connections_pending`</code>	> 5 pending

■ Things That Will Break in Production

- Querying Cassandra's ``order_activity`` by ``order_id`` directly — this is a PostgreSQL mistake that mirrors a Cassandra one: ``order_id`` is not indexed in Cassandra. Always resolve ``customer_id`` from PostgreSQL first.
- Forgetting ``@Transactional(readOnly=true)`` — Hibernate takes dirty-check snapshots for every entity loaded, wasting memory and CPU on read-only operations.
- Adding a NOT NULL column to ``orders`` without a DEFAULT — Flyway migration will fail on a table with millions of rows. Always add with a DEFAULT then drop the DEFAULT separately.
- Self-invocation of ``@Transactional`` methods — Spring AOP proxy is bypassed; the internal call runs in the caller's transaction regardless of ``propagation`` setting.
- Partition pruning lost when casting — ``WHERE CAST(created_at AS DATE) = '2026-06-01'`` defeats partition pruning. Always use range comparisons on the raw ``TIMESTAMP`` column.

4. Redis

4.1 What It Is & Why ShopVerse Uses It

Redis 7 is ShopVerse's in-memory data structure store, used for six distinct purposes: JWT session storage, application-level caching, distributed rate limiting, shopping cart state, pub/sub messaging, and distributed locking. Its sub-millisecond latency makes it ideal for hot paths like token validation (every request) and product cache lookups.

4.2 Memory Architecture & Persistence

```
Write path:
  SET key value
  ■
  ■■■→ In-memory hash table (instant)
  ■
  ■■■→ AOF (Append-Only File) – every write logged to disk
  OR
  RDB (Redis Database Snapshot) – periodic full snapshot
```

ShopVerse dev config: default (RDB snapshots)
 ShopVerse production recommendation: AOF with `fsync=everysec` for durability

4.3 Data Structure Usage in ShopVerse

Redis Key Namespace Map		
STRING	session:{token}	→ userId (JWT validation)
STRING	ratelimit:user:{id}	→ request count (rate limiter)
HASH	cart:{customerId}	→ {productId: quantity, ...}
LIST	viewed:{customerId}	→ [productId, ...] (last 10)
SET	tag:{tagName}	→ {productId, productId, ...}
SORTED SET	leaderboard	→ {productId: score, ...}
STREAM	orders:events	→ order status change stream
PUB/SUB	inventory:low	→ low stock alerts
STRING	products::{id}	→ serialized Product JSON
	customers::{id}	→ serialized Customer JSON
	orders::{id}	→ serialized Order JSON

4.4 Full Key Namespace Table

Key Pattern	Type	TTL	Set By	Read By	Evicted By
`session:{token}`	String	3600s (1hr)	`RedisSessionStore` on login	`JwtAuthenticationFilter` on every request	`RedisSessionStore` on logout (DEL) or TTL expiry
`cart:{customerId}`	Hash	Explicit EXPIRE	`ShoppingCartService.addItem()`	`ShoppingCartService.getCart()`	`ShoppingCartService.clear()`
`products::{id}`	String (JSON)	30 min	`CachedProductService.save()` (@CachePut)	`CachedProductService.findById()` (@Cacheable)	`CachedProductService.evict()` (@CacheEvict)
`customers::{id}`	String (JSON)	60 min	Customer update use case	Customer query service	Customer evict on update
`orders::{id}`	String (JSON)	5 min	`OrderCacheService` (manual RedisTemplate)	`OrderCacheService.getOrder()`	`OrderCacheService.evict()`
`ratelimit:user:{id}`	String (counter)	Set by Lua script (60s window)	`RateLimitAspect` via Lua INCR+EXPIRE	`RateLimitAspect` INCR check	TTL expiry
`leaderboard`	Sorted Set	No TTL	`ProductLeaderboard.ZADD`	`ZREVRANGE` leaderboard 0 9`	Manual ZREM
`viewed:{customerId}`	List	No TTL	`RecentlyViewedService.LPUSH`	`RecentlyViewedService.LRANGE`	`LTRIM` keeps last 10
`tag:{tagName}`	Set	No TTL	`ProductTagIndex.SADD`	`SINTER` tag:electronics tag:sale`	Manual SREM

Key Pattern	Type	TTL	Set By	Read By	Evicted By
`orders:events`	Stream	No TTL	`OrderEventStream` `.XADD`	`XREAD` consumer groups	Manual XTRIM
`inventory:low`	Pub/Sub channel	N/A (ephemeral)	`RedisPubSubPublisher`	`LowStockSubscriber`	N/A — no persistence
Redisson lock keys	String (Lua)	Lock TTL (10s)	`DistributedLockService.acquire()`	`DistributedLockService.isLocked()`	Lock release or TTL

4.5 Cache Configuration

Cache Name	TTL	Spring Annotation	Used In
`products`	30 min	`@Cacheable`, `@CachePut`, `@CacheEvict`	`CachedProductService`
`customers`	60 min	`@Cacheable`, `@CacheEvict`	Customer service
`orders`	5 min	Manual `RedisTemplate`	`OrderCacheService`
Default	10 min	—	Any cache not explicitly configured

@Cacheable / @CachePut / @CacheEvict Usage

Class	Method	Annotation	Cache	Key Expression
`CachedProductService`	`findById(id)`	`@Cacheable`	`products`	`#id`
`CachedProductService`	`save(product)`	`@CachePut`	`products`	`#result.id`
`CachedProductService`	`evict(id)`	`@CacheEvict`	`products`	`#id`
`CachedProductService`	`findAll()`	`@CacheEvict(allEntries=true)`	`products`	—
`ProductService`	`delete(id)`	`@Caching`	`products` + `product-search`	`#id`

Why OrderCacheService uses manual RedisTemplate instead of @Cacheable:

Orders have a short 5-minute TTL and need conditional caching logic (don't cache CANCELLED orders). `@Cacheable` doesn't support conditional eviction based on value state; manual `RedisTemplate.opsForValue()` gives full control.

4.6 JWT Session Store Pattern

Login flow:

```
POST /api/auth/login → JwtTokenProvider.generateToken()
    ■ creates JWT (sub=customerId, exp=15min)
```

■

▼

```
RedisSessionStore.store(token, userId)
```

```
    ■ SET session:{token_sha256_hash} {userId} EX 3600
```

▼

Returns JWT to client

Request validation flow:

```
GET /api/orders → Authorization: Bearer {token}
```

■

▼

```
JwtAuthenticationFilter → JwtTokenProvider.validateToken()
```

```
    ■ checks signature + expiry (stateless)
```

■

▼

```
RedisSessionStore.isValid(token)
```

```
    ■ GET session:{token_sha256_hash}
```

```
    ■ NULL → token blacklisted or expired → 401
```

```
    ■ EXISTS → valid → continue
```

Logout flow:

```
POST /api/auth/logout
```

■

▼

```
RedisSessionStore.invalidate(token)
```

```
    ■ DEL session:{token_sha256_hash}
```

```
    → Token now returns NULL – effectively blacklisted
```

4.7 Rate Limiter Lua Script

The Lua script ensures INCR + EXPIRE is atomic (no race condition between two operations):

```
local key = KEYS[1]          -- e.g. "ratelimit:user:42"
local limit = ARGV[1]        -- e.g. 60
local window = ARGV[2]       -- e.g. 60 (seconds)

local count = redis.call('INCR', key)
if count == 1 then
    redis.call('EXPIRE', key, window) -- set TTL only on first increment
end
if count > tonumber(limit) then
    return 0 -- rate limited
end
return 1 -- allowed
```

Configured limits: `@RateLimit(perMinute=60)` on `OrderController.placeOrder()` and `AuthController.login()`.

4.8 Redisson Distributed Lock

```
FlashSaleService.buyNow(productId):
```

Node A

```
■  
■ getLock("flash-sale:553")  
■ tryLock(timeout=10s)  
■  
■ ← LOCK ACQUIRED  
■  
■ [purchase logic runs]  
■  
■ unlock()  
■ ← LOCK RELEASED
```

Node B

```
■  
■ getLock("flash-sale:553")  
■ tryLock(timeout=10s)  
■  
■ ← WAITING (lock held by A)  
■  
■  
■  
■  
■  
■ ← LOCK ACQUIRED (now)  
■ [purchase logic runs]  
■ unlock()
```

Under the hood: Lua script in Redis

```
SET flash-sale:553 {nodeId:threadId} PX 10000 NX
```

(SET if Not eXists, expiry 10000ms)

Returns 1 if acquired, 0 if already held.

Watchdog thread extends TTL if holder is still alive.

4.9 Eviction Policies

Policy	Description	When to Use
<code>`noeviction`</code>	Returns error when memory full	Never for cache use cases
<code>`allkeys-lru`</code>	Evicts least-recently-used keys (any key)	**ShopVerse recommendation** — general cache
<code>`volatile-lru`</code>	Evicts LRU keys that have TTL set	When some keys must never be evicted
<code>`allkeys-lfu`</code>	Evicts least-frequently-used keys	When access frequency matters more than recency
<code>`volatile-ttl`</code>	Evicts keys with shortest remaining TTL	When you want expired-soon keys evicted first

ShopVerse `docker-compose.yml` sets `--maxmemory-policy allkeys-lru`.

4.10 Monitoring Signals

Signal	Redis Command	Alert
Memory usage	<code>`INFO memory`</code> → <code>`used_memory_rss`</code>	> 80% of <code>`maxmemory`</code>
Cache hit rate	<code>`INFO stats`</code> → <code>`keyspace_hits / (keyspace_hits + keyspace_misses)`</code>	< 80% hit rate
Eviction rate	<code>`INFO stats`</code> → <code>`evicted_keys`</code>	Rapidly increasing
Connected clients	<code>`INFO clients`</code> → <code>`connected_clients`</code>	> 100
Blocked clients	<code>`INFO clients`</code> → <code>`blocked_clients`</code>	Any > 0

■ Things That Will Break in Production

- Using `KEYS \*` in production — O(N) operation, blocks Redis single thread. Use `SCAN` instead.
- Pub/Sub channel `inventory:low` has no persistence — if the subscriber is offline when a message is published, the message is lost. Use Redis Streams for durability.
- Cart stored as a Hash without expiry — abandoned carts grow indefinitely. Always set `EXPIRE` on cart keys.
- Redisson lock without `finally` block — if the holder crashes before `unlock()`, the lock TTL (10s) must expire before another node can acquire it. Always release in `finally`.
- Spring Cache `@CacheEvict(allEntries=true)` on a large product set — evicts all product cache entries simultaneously, causing a cache stampede on the next read burst. Mitigate with Redisson lock on cache rebuild.

5. MongoDB

5.1 What It Is & Why ShopVerse Uses It

MongoDB 7 is ShopVerse's document store, used for product reviews and flexible product specifications. Reviews are stored in MongoDB because they are unbounded in number, schema-flexible (pros/cons arrays, nested metadata), and accessed independently of the core product catalog. MongoDB's aggregation pipeline makes average rating computation efficient without a JOIN.

5.2 Document Model vs Relational

Relational (PostgreSQL):

products table: id, name, price
reviews table: id, product_id FK, rating, comment
→ JOIN required to get product with reviews

MongoDB:

reviews collection: { productId, rating, comment, pros: [], cons: [] }
products collection: { name, price, specs: [{key,value},...], avgRating }
→ Each document is self-contained; no JOIN needed for common reads

5.3 ShopVerse Collections

reviews (`ReviewDocument`)

Field	BSON Type	Description
`_id`	ObjectId	Auto-generated MongoDB ID
`productId`	String	References PostgreSQL `products.id`
`customerId`	Long	References PostgreSQL `customers.id`
`customerName`	String	Denormalised for display without a lookup

Field	BSON Type	Description
`rating`	Int	1–5 star rating
`comment`	String	@TextIndexed — full-text searchable
`pros`	Array	Embedded array of positive points
`cons`	Array	Embedded array of negative points
`verifiedPurchase`	Boolean	Whether customer bought this product
`createdAt`	Date	Review submission timestamp

Indexes on reviews:

- `{ productId: 1 }` — for `db.reviews.find({productId: '1'})` lookups
- `{ $text: {comment: "text"} }` — for full-text search on review comments

products (`ProductDocument`)

Field	BSON Type	Description
`_id`	String	Mirrors PostgreSQL `products.id`
`name`	String	@TextIndexed (boosted)
`description`	String	@TextIndexed
`category`	String	Category for filtering
`price`	Decimal128	Product price
`stockQuantity`	Int	Denormalised stock (synced from PostgreSQL)
`active`	Boolean	Product visibility flag
`specifications`	Array<{key, value}>	**Embedded** — flexible key-value specs
`avgRating`	Double	**Computed pattern** — updated on each new review
`hasMoreReviews`	Boolean	**Outlier pattern** — true when review count > 1000
`nameSuggest`	Completion	Elasticsearch completion suggester field
`createdAt`	Date	Index timestamp

Why specs are embedded but reviews are referenced:

- Specs: bounded size (< 20 items), always accessed with the product, never queried independently
- Reviews: unbounded growth, queried independently (pagination, aggregation), would bloat the product document

DailyOrderBucket (`DailyOrderBucket` — Bucket Pattern)

```
{
  "date": "2026-06-13",
  "orders": [
    {"orderId": 152, "customerId": 202, "total": 202.00},
    {"orderId": 153, "customerId": 202, "total": 4998.00}
  ],
  "count": 2,
  "totalRevenue": 5200.00
}
```

Populated by a [@Scheduled](#) job. Avoids aggregating millions of order rows for daily revenue dashboards.

5.4 Aggregation Pipeline — Average Rating

`ReviewService.getAverageRating(productId)` runs:

```
Stage 1: $match { productId: '553' }
          ↓ Filters to reviews for product 553 only
Stage 2: $group { _id: '$productId', avgRating: { $avg: '$rating' }, count: { $sum: 1 } }
          ↓ Groups all matching reviews into one document with average
Stage 3: $sort { avgRating: -1 }
          ↓ Orders results (useful for multi-product calls)

Result: { _id: '553', avgRating: 4.3, count: 47 }
```

The result is then used to update `ProductDocument.avgRating` (computed pattern) so future reads don't rerun the aggregation.

5.5 Monitoring Signals

Signal	Command	Alert
Slow queries	<code>`db.setProfilingLevel(1, {slowms: 100})`</code>	Any query > 100ms
Index usage	<code>`db.reviews.explain("executionStats").find(...)`</code>	<code>`COLLSCAN`</code> on large collection
Collection size	<code>`db.reviews.stats().size`</code>	Growing unboundedly
Replica lag	<code>`rs.status()`</code>	Secondary lag > 10s

■ Things That Will Break in Production

- Missing index on ``reviews.productId`` — every ``getReviews(productId)`` does a full collection scan as the collection grows.
- ``avgRating`` computed field going stale — if ``ReviewService.submitReview()`` fails after saving the review but before updating ``avgRating``, the displayed rating is wrong until the next review.
- ``hasMoreReviews`` flag never set — the outlier pattern is defined but if the flag-setting logic isn't triggered at 1000 reviews, the product document bloats indefinitely.
- **No schema validation** — MongoDB accepts any document shape. A miscoded producer can silently insert malformed reviews with missing ``rating`` fields, breaking aggregation.
- ``productId`` stored as `String` not `ObjectId` — cross-referencing with PostgreSQL ``products.id`` (Long) requires careful type handling; a `String "553" ≠ Long 553` in typed comparisons.

6. Cassandra

6.1 What It Is & Why ShopVerse Uses It

Apache Cassandra 4.1 is ShopVerse's wide-column store for the order activity log. Every order state transition (`ORDER_PLACED`, `ORDER_CONFIRMED`, `ORDER_SHIPPED`, etc.) is appended as an immutable event. Cassandra was chosen because it offers **linear write scalability** and **time-series optimised reads by customer** — exactly the access pattern for "show me all activity for customer 202".

6.2 Ring Architecture

Cassandra Cluster (ShopVerse dev: 1 node, prod: 3+ nodes)

```
      Node A (tokens 0-33%)
    /           \
Node C (tokens 67-100%) Node B (tokens 34-66%)
    \           /
    (ring continues...)
```

- Data is distributed by consistent hashing of the partition key
- Each node owns a range of the token ring
- Replication: each row is stored on N nodes (RF=1 in dev, RF=3 in prod)
- No single point of failure in a multi-node cluster

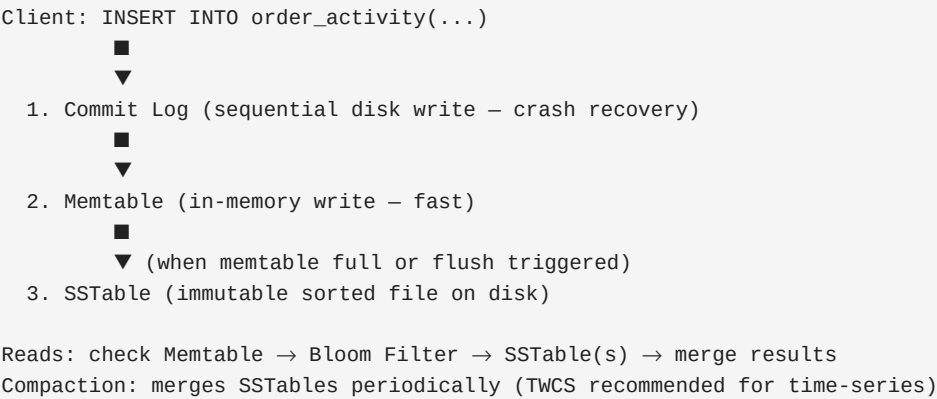
6.3 Consistency Levels in ShopVerse

Operation	Consistency Level	Meaning	ShopVerse Rationale
Write (order activity)	`QUORUM`	Majority of nodes must acknowledge	Ensures data not lost if one node fails immediately after write
Read (activity log)	`LOCAL_ONE`	One node in local DC must respond	Fast reads; staleness acceptable for activity log display

Tradeoff table:

Level	Write Speed	Read Speed	Durability	Use When
ONE	Fastest	Fastest	Lowest	Non-critical metrics
QUORUM	Medium	Medium	High	**ShopVerse writes**
ALL	Slowest	Slowest	Highest	Financial transactions
LOCAL_ONE	Fast	Fastest	Low-Medium	**ShopVerse reads**

6.4 Write Path



6.5 order_activity Table Schema

```
USE shopverse;

CREATE TABLE order_activity (
  customer_id BIGINT,
  event_time  TIMESTAMP,
  event_id    UUID,
  order_id    BIGINT,
  event_type  TEXT,
  details     TEXT,
  PRIMARY KEY ((customer_id), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC, event_id ASC)
AND compaction = {'class': 'TimeWindowCompactionStrategy',
                  'compaction_window_unit': 'DAYS',
                  'compaction_window_size': 1};
```

PRIMARY KEY breakdown:

Component	Column	Role	Rationale
Partition key	`customer_id`	Determines which node(s) store the row	All activity for one customer is co-located on one node — fast customer timeline queries
Clustering key 1	`event_time DESC`	Physical sort order within partition	Newest events first without ORDER BY in query
Clustering key 2	`event_id` UUID	Uniqueness within same millisecond	Two events at exactly the same timestamp are distinguishable
Regular column	`order_id`	Filterable only in memory	NOT the partition key — querying by `order_id` alone requires ALLOW FILTERING (full cluster scan)

6.6 Events Written to order_activity

event_type	Written By	When	Java Class
`ORDER_PLACED`	`PlaceOrderUseCase`	After PostgreSQL commit	`OrderActivityRepository.save()`
`ORDER_CONFIRMED`	`UpdateOrderStatusUseCase`	Admin confirms	`OrderActivityRepository.save()`
`ORDER_SHIPPED`	`UpdateOrderStatusUseCase`	Admin ships	`OrderActivityRepository.save()`
`ORDER_DELIVERED`	`UpdateOrderStatusUseCase`	Admin delivers	`OrderActivityRepository.save()`
`ORDER_CANCELLED`	`UpdateOrderStatusUseCase`	Cancel action	`OrderActivityRepository.save()`
`PAYMENT_CAPTURED`	`PaymentService`	Payment success	`OrderActivityRepository.save()`

6.7 Two-Step Order Activity Lookup

API call: GET /api/orders/152/activity

Step 1 – PostgreSQL lookup (JpaOrderRepository):

```
SELECT customer_id FROM orders WHERE id = 152  
→ returns customer_id = 202
```

Step 2 – Cassandra query (OrderActivityRepository):

```
SELECT * FROM shopverse.order_activity  
WHERE customer_id = 202 ← uses partition key ✓  
→ returns all activity rows for customer 202
```

Step 3 – Java stream filter (in OrderController / use case):

```
.stream()  
.filter(a -> a.getOrderId().equals(152L))  
.collect(toList())  
→ returns only activity for order 152
```

■ NEVER DO THIS:

```
SELECT * FROM order_activity WHERE order_id = 152  
→ ALLOW FILTERING error – Cassandra would scan the entire cluster
```

6.8 Monitoring Signals

Signal	Tool	Alert
Read/write latency	`nodetool tpstats`	p99 > 10ms
Pending compactions	`nodetool compactionstats`	> 100 pending
Dropped messages	`nodetool tpstats`	Any dropped mutations

Signal	Tool	Alert
Heap pressure	<code>`nodetool info`</code>	Heap usage > 75%
Hinted handoff	<code>`nodetool tpstats`</code>	High hinted handoff rate = node was down

■ Things That Will Break in Production

- Querying by ``order_id`` directly — ``WHERE order_id = 152`` always fails with ALLOW FILTERING. Always use the two-step pattern.
- Using ``SimpleStrategy`` in production — it doesn't understand data centre topology. Use ``NetworkTopologyStrategy`` with a replication factor per DC.
- RF=1 in production — one node failure loses data permanently. Use RF=3 minimum.
- Cassandra write after PostgreSQL commit with no retry — if Cassandra is momentarily unavailable, the activity event is lost with no retry. Wrap the Cassandra write in a retry with exponential backoff.
- TWCS compaction window mismatch — if the compaction window (1 day) doesn't match your query patterns (e.g., queries spanning weeks), read amplification increases dramatically.

7. Elasticsearch

7.1 What It Is & Why ShopVerse Uses It

Elasticsearch 8.13 powers ShopVerse's product search — full-text search, relevance scoring, autocomplete, and fuzzy matching. PostgreSQL's `ILIKE` query cannot score results by relevance or handle synonyms/typos. Elasticsearch's inverted index gives sub-10ms search across millions of products.

7.2 Inverted Index

Documents:

Doc 1: "Laptop Pro – high performance laptop for professionals"
 Doc 2: "Gaming Laptop – powerful laptop with RGB keyboard"

Inverted Index after tokenisation + lowercase + stop word removal:

```
"laptop"    → [Doc 1, Doc 2]
"pro"       → [Doc 1]
"high"      → [Doc 1]
"performance" → [Doc 1]
"gaming"    → [Doc 2]
"powerful"  → [Doc 2]
"rgb"       → [Doc 2]
"keyboard"  → [Doc 2]
```

Query: "laptop"

```
→ look up "laptop" in index → [Doc 1, Doc 2]
→ O(1) lookup, not O(N) table scan
```

7.3 Index → Shard → Segment Hierarchy

Index: products

■■■■ Shard 0 (primary, on Node 1)
■ ■■■■ Segment 1 (immutable SST-like file)
■ ■■■■ Segment 2
■ ■■■■ Segment N ← merged periodically
■■■■ Shard 1 (primary, on Node 2)
■■■■ Shard 2 (primary, on Node 3)

Dev: 1 shard, 0 replicas (single node)

Prod recommendation: 1 shard per 30GB of data, 1 replica per shard

7.4 ShopVerse products Index Mapping

```
{
  "mappings": {
    "properties": {
      "name": { "type": "text", "analyzer": "shopverse_analyzer", "boost": 3 },
      "description": { "type": "text", "analyzer": "shopverse_analyzer" },
      "sku": { "type": "keyword" },
      "category": { "type": "keyword" },
      "price": { "type": "scaled_float", "scaling_factor": 100 },
      "stockQuantity": { "type": "integer" },
      "active": { "type": "boolean" },
      "avgRating": { "type": "float" },
      "tags": { "type": "keyword" },
      "nameSuggest": { "type": "completion" },
      "createdAt": { "type": "date" }
    }
  },
  "settings": {
    "analysis": {
      "analyzer": {
        "shopverse_analyzer": {
          "tokenizer": "standard",
          "filter": ["lowercase", "stop", "synonym"]
        }
      }
    },
    "refresh_interval": "30s"
  }
}
```

Field type choices:

- name as text with boost 3 — relevance-scored, weighted 3x over description
- sku as keyword — exact match only, not analysed
- nameSuggest as completion — special type for autocomplete suggester

7.5 ProductSearchService Query Structure

```
bool query {
  must: [
    multi_match {
      query: "laptop",
      fields: ["name^3", "description^1"],
      fuzziness: AUTO    ← handles typos
    }
  ],
  filter: [
    range { price: { gte: minPrice, lte: maxPrice } },
    term { category: "electronics" },
    terms { tags: ["sale", "new"] }
  ],
  should: [
    term { stockQuantity: { gt: 0 } } ← boosts in-stock items
  ]
}
```

`searchWithBoost()` wraps this in a `function_score`:

```
function_score {
  query: [above bool query],
  functions: [
    { filter: { range: { avgRating: { gt: 4 } } }, weight: 1.5 },
    { filter: { range: { stockQuantity: { gt: 0 } } }, weight: 2.0 }
  ],
  score_mode: "multiply"
}
```

7.6 Autocomplete Flow

User types "lap" in search box

```
■
▼ GET /api/v1/search/suggest?q=lap
■
▼ SearchController.autocomplete()
■
▼ Elasticsearch suggest query:
{
  "suggest": {
    "product-suggest": {
      "prefix": "lap",
      "completion": {
        "field": "nameSuggest",
        "size": 5
      }
    }
  }
}
■
▼ Returns: ["Laptop Pro", "Laptop Gaming", "Laptop Stand", ...]
```

`nameSuggest` is populated in `ProductSyncService.indexProduct()`:

```
doc.setSuggest(new Completion(new String[]{ product.getName() }));
```

7.7 Sync Mechanism — Two Paths

Path 1: TransactionEventListener (primary, always active)

```
CreateProductUseCase.execute()
→ eventPublisher.publish(ProductEvent.ProductCreated)
→ [PostgreSQL transaction commits]
→ ProductSyncService.onProductCreated() [AFTER_COMMIT, @Async]
→ productRepository.findById() [re-reads from DB]
→ searchRepo.save(ProductDocument)
→ [Elasticsearch indexed]
```

Path 2: Debezium CDC (alternative, @ConditionalOnProperty)

```
PostgreSQL WAL → pgoutput replication slot
→ Debezium embedded engine
→ ProductChangeEventHandler
→ ProductSearchService.upsert() or delete()
→ [Elasticsearch indexed]
```

Delete flow (via ProductSyncService.onProductUpdated):

```
if (event.field().equals("deleted")) {
    searchRepo.deleteById(productId) ← removes from ES
} else {
    indexProduct(product)             ← re-indexes updated product
}
```

7.8 BulkProductIndexer — Reindex Flow

reindexAll() triggered by admin endpoint:

1. esOps.indexOps(ProductDocument.class).delete()
→ Drops existing "products" index
2. esOps.indexOps(ProductDocument.class).createWithMapping()
→ Recreates with correct mappings (especially completion field)
3. Loop (page=0, pageSize=100):
batch = productRepository.findAll(page, pageSize)
batch.forEach(this::indexProduct)
page++
until batch.size() < pageSize
4. Logs: "Reindex complete — N products indexed"

7.9 Monitoring Signals

Signal	Endpoint	Alert
Cluster health	<code>`GET /_cluster/health`</code>	Status <code>`red`</code> (data loss) or <code>`yellow`</code> (unassigned shards)
Search latency	<code>`GET /_cat/indices?v`</code> query time	p99 > 50ms
Indexing rate	<code>`GET /_stats`</code>	Drop to 0 during active product saves
Unassigned shards	<code>`GET /_cat/shards`</code>	Any UNASSIGNED
JVM heap	<code>`GET /_nodes/stats/jvm`</code>	> 75% heap used

■ Things That Will Break in Production

- ``refresh_interval: 30s`` — newly indexed products won't appear in search for up to 30 seconds. Acceptable for bulk loads, not for real-time product creation.
- Dropping and recreating the index during `reindexAll()` — search returns 0 results during the reindex window. Use an alias (``products-v2``) and swap atomically.
- ``ProductDocument`` and ``ProductSearchDocument`` diverging — two separate classes representing the same data will drift; a field added to one may be missed in the other.
- Completion suggester requires exact field name ``nameSuggest`` of type ``completion`` — adding it after index creation requires a reindex; you cannot change field types in-place.
- ``@TransactionalEventListener(AFTER_COMMIT)`` failure — if the async indexing thread fails (ES down), the product is saved to PostgreSQL but not indexed. No retry mechanism is configured.

8. Neo4j

8.1 What It Is & Why ShopVerse Uses It

Neo4j 5 is ShopVerse's graph database, used for product recommendations via collaborative filtering. The "customers who bought this also bought" pattern requires traversing relationships across customers and products — a query that would require multiple self-joins in SQL but is a natural two-hop graph traversal in Cypher.

8.2 ShopVerse Graph Schema

Graph Model:

```
(:Customer {id, email})
  ■
  ■ [PURCHASED {orderId, purchasedAt, quantity}]
  ■
  ▼
(:Product {id, sku, name})
```

Example graph:

```
(Customer:1 "Alice") ■■[PURCHASED {orderId:52}]■■■ (Product:553 "Laptop Pro")
(Customer:1 "Alice") ■■[PURCHASED {orderId:52}]■■■ (Product:10 "Bluetooth Speaker")
(Customer:2 "Bob")   ■■[PURCHASED {orderId:68}]■■■ (Product:553 "Laptop Pro")
(Customer:2 "Bob")   ■■[PURCHASED {orderId:68}]■■■ (Product:6 "Wireless Charger")
```

Recommendation for Alice:

```
Alice bought 553 → Bob also bought 553 → Bob also bought 6
→ Recommend Product 6 "Wireless Charger" to Alice
```

8.3 Node & Relationship Classes

Class	Type	@Node/@RelationshipProperties	Fields
`ProductNode`	Node	`@Node("Product")`	`id: Long`, `sku: String`, `name: String`
`CustomerNode`	Node	`@Node("Customer")`	`id: Long`, `email: String`, `purchasedProducts: List`
`PurchasedRelationship`	Relationship	`@RelationshipProperties`	`orderId: Long`, `purchasedAt: Instant`, `quantity: int`

`CustomerNode.purchasedProducts` is annotated:

```
@Relationship(type = "PURCHASED", direction = OUTGOING)
List<PurchasedRelationship> purchasedProducts;
```

8.4 Collaborative Filtering Query — Annotated

```
MATCH (c:Customer {id: $customerId}) -- Find the target customer
      -[:PURCHASED]->                -- Who has purchased products
      (p:Product)                    -- (intermediate products)
      <-[:PURCHASED]-                -- Which other customers also purchased
      (other:Customer)               -- (those other customers)
      -[:PURCHASED]->                -- Who also purchased
      (rec:Product)                  -- (recommendation candidates)
WHERE NOT (c)-[:PURCHASED]->(rec)    -- Exclude products Alice already owns
RETURN rec, COUNT(*) AS score         -- Score by how many co-buyers bought it
ORDER BY score DESC
LIMIT 10
```

8.5 Neo4jSyncListener — Graph Population

OrderShipped event (or after order completes):

```
Neo4jSyncListener.onOrderCompleted(OrderPlacedEvent)
  @Async
  @TransactionalEventListener(phase = AFTER_COMMIT)
  ■
  ▼
  For each item in the order:
  recommendationRepository.createPurchasedRelationship(
    customerId, productId, orderId, quantity, Instant.now()
  )
  ■
  ▼
  Cypher: MERGE (c:Customer {id: $customerId})
          MERGE (p:Product {id: $productId})
          MERGE (c)-[r:PURCHASED {orderId: $orderId}]->(p)
          ON CREATE SET r.purchasedAt = $now, r.quantity = $qty
```

If Neo4j is down at commit time:

- @Async means exception is swallowed (fire-and-forget)
- Recommendation graph will be stale until next sync
- No retry configured – production should add a retry mechanism

8.6 Graceful Degradation

When a customer or product has no graph data:

```
RecommendationController.getRecommendations(productId):
  try {
    return recommendationRepository.findRecommendations(productId)
  } catch (Exception e) {
    return Collections.emptyList() ← returns empty, UI shows nothing
  }
```

New product (no PURCHASED relationships yet):

Cypher returns 0 rows → empty list → UI falls back to "popular products"

8.7 Monitoring Signals

Signal	Tool	Alert
Query response time	Neo4j Browser → Query log	Cypher query > 1s
Relationship count	<code>`MATCH ()-[r:PURCHASED]->() RETURN COUNT(r)`</code>	Not growing after orders placed
JVM heap	<code>`:sysinfo`</code> in Neo4j Browser	> 80% heap
Lock contention	Neo4j logs	Any deadlock warnings

■ Things That Will Break in Production

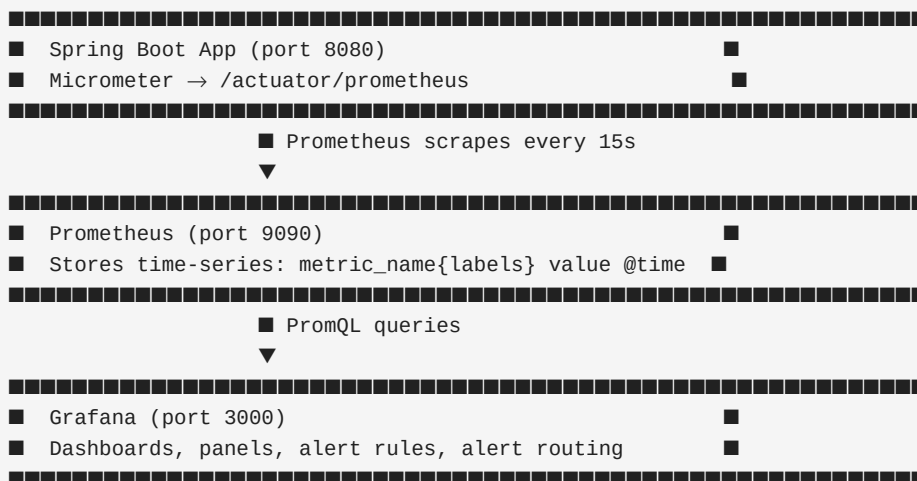
- `@Async`` sync listener with no retry — if Neo4j is briefly unavailable post-commit, purchase relationships are permanently lost from the graph. Orders won't affect recommendations.
- `MERGE`` without unique constraints — if ``Customer {id}`` and ``Product {id}`` don't have uniqueness constraints, parallel transactions can create duplicate nodes.
- Collaborative filtering returns nothing for new users — cold start problem. No fallback to popularity-based recommendations is implemented.
- No index on ``Customer.id`` and ``Product.id`` — `MATCH (c:Customer {id: $id})`` does a full node scan without an index. Create range indexes: `CREATE INDEX FOR (c:Customer) ON (c.id)``.
- `PURCHASED`` relationship has no uniqueness guard — replaying an order event twice creates duplicate ``PURCHASED`` relationships, inflating recommendation scores. The `MERGE`` on ``orderId`` prevents this only if ``orderId`` is al

9. Prometheus & Grafana

9.1 What They Are & Why ShopVerse Uses Them

Prometheus is a pull-based time-series metrics database. Grafana is a dashboard and alerting frontend. Together they give ShopVerse real-time visibility into application health, throughput, latency, and infrastructure state. Spring Boot's Micrometer integration exposes metrics at `/actuator/prometheus` that Prometheus scrapes every 15 seconds.

9.2 Pull-Based Architecture



9.3 Metric Types

Type	Description	ShopVerse Example
Counter	Monotonically increasing count — never decreases	<code>`failedPaymentCounter`</code> — total payment failures since startup

Type	Description	ShopVerse Example
Gauge	Current value — can go up or down	<code>`activeCartGauge`</code> — number of active shopping carts right now
Histogram	Distribution of values in configurable buckets	<code>`shopverse_order_placement_seconds`</code> — latency of <code>`placeOrder()`</code>
Timer	Histogram + counter for timing operations	<code>`@Timed("shopverse.product.search")`</code> on <code>`ProductSearchService.search()`</code>
Summary	Pre-computed quantiles (not recommended — can't aggregate)	Not used in ShopVerse

9.4 ShopVerse Custom Metrics

Metric Name	Type	Registered In	Labels	Measures
<code>`shopverse_order_placement_seconds`</code>	Timer	<code>`OrderApplicationService`</code>	<code>`status`</code> (success/failure)	End-to-end latency of order placement including DB, cache, Kafka publish
<code>`shopverse_failed_payment_total`</code>	Counter	<code>`PaymentService`</code>	<code>`reason`</code>	Count of payment failures by reason code
<code>`shopverse_active_carts`</code>	Gauge	<code>`CartService`</code>	—	Number of non-empty shopping carts currently in Redis
<code>`shopverse_product_search_seconds`</code>	Timer	<code>`ProductSearchService`</code> via <code>`@Timed`</code>	—	Elasticsearch query latency

9.5 Key PromQL Queries

```
# HTTP request rate (per second over 5 min window)
rate(http_server_requests_seconds_count[5m])

# p99 latency across all endpoints
histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m]))

# Order placement p99 latency
histogram_quantile(0.99, rate(shopverse_order_placement_seconds_bucket[1m]))

# JVM heap utilisation (%)
jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes{area="heap"} * 100

# HikariCP pool saturation
hikaricp_connections_pending / hikaricp_connections_max * 100

# Kafka consumer lag (total across all partitions)
sum(kafka_consumer_records_lag) by (topic)

# Payment failure rate per minute
rate(shopverse_failed_payment_total[1m])
```

9.6 Grafana Dashboard Panels

Panel	PromQL	Alert Threshold
Order Throughput	<code>`rate(http_server_requests_seconds_count{uri="/api/v1/orders",method="POST"}[1m])`</code>	< 0 (drop to zero)
p99 Latency	<code>`histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m]))`</code>	> 500ms
Error Rate	<code>`rate(http_server_requests_seconds_count{status=~"5.."}[5m]) / rate(http_server_requests_seconds_count[5m])`</code>	> 1%
DB Connection Pool	<code>`hikaricp_connections_pending`</code>	> 5 pending
Kafka Consumer Lag	<code>`sum(kafka_consumer_records_lag) by (topic)`</code>	> 1000
JVM Heap Used	<code>`jvm_memory_used_bytes{area="heap"}`</code>	> 800MB
Active Carts	<code>`shopverse_active_carts`</code>	— (informational)

9.7 Alert Rules (alert_rules.yml)

Alert Name	Condition	Severity	Operational Meaning
<code>`HighLatency`</code>	p99 > 500ms for 5 min	warning	DB slowness, pool exhaustion, or ES degradation
<code>`HighErrorRate`</code>	5xx > 1% for 2 min	critical	Service is broken — needs immediate investigation
<code>`KafkaConsumerLag`</code>	lag > 1000 on any topic	warning	Consumer is behind; messages piling up
<code>`DBPoolSaturation`</code>	pending > 5 for 1 min	critical	All DB connections in use; new requests queuing
<code>`ReplicationLag`</code>	PostgreSQL lag > 50MB	warning	Replica is behind; read queries may see stale data
<code>`DLTHasMessages`</code>	DLT offset advancing	critical	Kafka messages permanently failed after all retries

9.8 Monitoring Signals

Signal	Query	Alert
Prometheus scrape failures	<code>`up{job="shopverse"}`</code>	<code>`up == 0`</code>

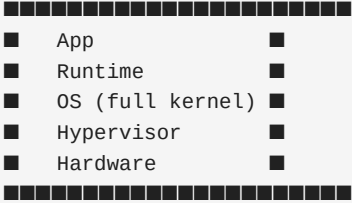
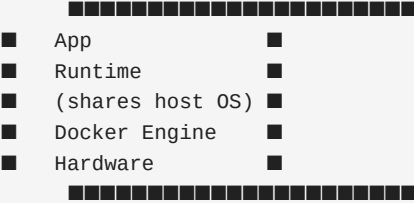
Signal	Query	Alert
Metrics endpoint down	<code>`curl localhost:8080/actuator/prometheus`</code> fails	Immediate

■ Things That Will Break in Production

- Counter metric resets on app restart — Prometheus rates handle this with ``rate()`` (detects resets), but raw counter values are misleading after a restart.
- High-cardinality labels — adding ``userId`` or ``orderId`` as a Micrometer label creates millions of time series and causes Prometheus OOM.
- Grafana data source pointing to ``localhost:9090`` — inside Docker, Grafana must use ``shopverse-prometheus:9090``, not ``localhost``.
- No alertmanager configured — ``alert_rules.yml`` firing has no effect without Alertmanager routing alerts to PagerDuty/Slack.
- ``management.tracing.sampling.probability=1.0`` in production — 100% trace sampling at high QPS overwhelms Zipkin and adds latency. Use 0.1 (10%) in production.

10. Docker & Networking

10.1 Container vs VM

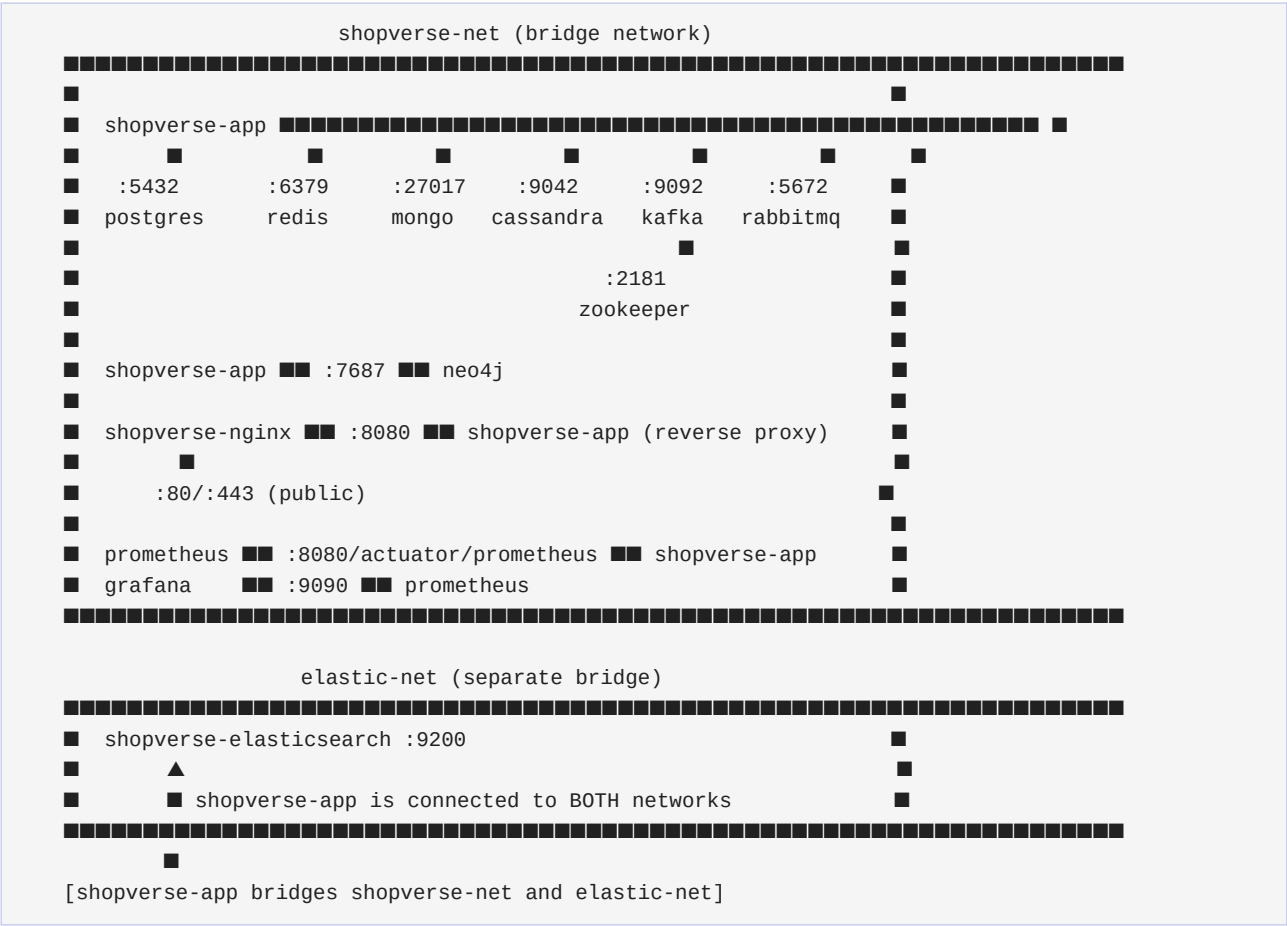
Virtual Machine:	Docker Container:
	
~1GB RAM overhead	~10MB RAM overhead
~30s boot time	~1s boot time
Full isolation (kernel)	Process isolation (namespaces)

10.2 ShopVerse docker-compose Service Inventory

Container Name	Image	Ports	Volumes	Health Check
<code>`shopverse-postgres`</code>	<code>`postgres:16`</code>	<code>`5432`</code>	<code>`postgres-data:/var/lib/postgresql/data`</code>	<code>`pg_isready -U shopverse`</code>
<code>`shopverse-redis`</code>	<code>`redis:7-alpine`</code>	<code>`6379`</code>	<code>`redis-data:/data`</code>	<code>`redis-cli ping`</code>
<code>`shopverse-mongo`</code>	<code>`mongo:7`</code>	<code>`27017`</code>	<code>`mongo-data:/data/db`</code>	<code>`mongosh --eval "db.runCommand({ping:1})"`</code>
<code>`shopverse-cassandra`</code>	<code>`cassandra:4.1`</code>	<code>`9042`</code>	<code>`cassandra-data:/var/lib/cassandra`</code>	<code>`cqlsh -e "DESCRIBE KEYSPACES"`</code>
<code>`shopverse-elasticsearch`</code>	<code>`elasticsearch:8.13.0`</code>	<code>`9200, 9300`</code>	<code>`es-data:/usr/share/elasticsearch/data`</code>	<code>`curl localhost:9200/_cluster/health`</code>

Container Name	Image	Ports	Volumes	Health Check
`shopverse-kafka`	`confluentinc/cp-kafka:7.6.0`	`9092, 29092`	—	`kafka-topics --list`
`shopverse-rabbitmq`	`rabbitmq:3.13-management`	`5672, 15672`	—	`rabbitmq-diagnostics ping`
`shopverse-neo4j`	`neo4j:5`	`7474, 7687`	`neo4j-data:/data`	`curl localhost:7474`
`shopverse-prometheus`	`prom/prometheus`	`9090`	`./infra/monitoring:/etc/prometheus`	`curl localhost:9090/-/healthy`
`shopverse-grafana`	`grafana/grafana`	`3000`	`grafana-data:/var/lib/grafana`	`curl localhost:3000/api/health`
`shopverse-nginx`	`nginx:alpine`	`80, 443`	`./nginx/nginx.conf:/etc/nginx/nginx.conf`	`curl localhost/health`
`shopverse-jenkins`	`jenkins/jenkins:lts`	`8090`	`jenkins-data:/var/jenkins_home`	`curl localhost:8090/login`
`shopverse-app`	`shopverse-web:latest` (built)	`8080`	—	`curl localhost:8080/actuator/health`
`shopverse-ui`	`shopverse-ui:latest` (built)	`5173`	—	`curl localhost:5173`

10.3 Network Topology



Why Elasticsearch is on a separate network: Security isolation. Elasticsearch 8.x has security features enabled by default; isolating it on its own network prevents other containers from making unauthorized direct requests.

10.4 Multi-Stage Dockerfile — Annotated

```
# ■■■ Stage 1: Build ████████████████████████████████████████████████████  
FROM maven:3.9-eclipse-temurin-21 AS builder  
# Large image with JDK + Maven – only used during build, not in final image  
  
WORKDIR /build  
  
COPY pom.xml .  
RUN mvn dependency:go-offline -q  
# Download dependencies BEFORE copying source – Docker layer cache  
# If source changes but pom.xml doesn't, this layer is reused (fast CI)  
  
COPY src ./src  
RUN mvn -B package -DskipTests  
# Build the fat JAR  
  
# ■■■ Stage 2: Runtime ████████████████████████████████████████████████████  
FROM eclipse-temurin:21-jre-jammy AS runtime  
# Minimal JRE image (~200MB vs ~600MB for JDK) – no compiler, no Maven  
  
RUN addgroup -S shopverse && adduser -S shopverse -G shopverse  
# Non-root user – if container is compromised, attacker has limited OS access  
  
WORKDIR /app  
COPY --from=builder /build/target/shopverse-web-*.jar app.jar  
# Copy ONLY the built JAR from the builder stage – source code not included  
  
USER shopverse  
# Drop root privileges before starting the app  
  
EXPOSE 8080  
  
ENTRYPOINT ["java",  
  
"-XX:+UseG1GC",           # G1 garbage collector – good for heap > 4GB and low pause targets  
"-Xms512m",               # Start heap at 512MB – avoids slow initial growth  
"-Xmx1g",                 # Max heap 1GB – prevents OOM in constrained containers  
"-XX:MaxGCPauseMillis=200", # Target GC pauses < 200ms  
"-XX:+HeapDumpOnOutOfMemoryError", # Capture heap dump on OOM for analysis  
"-XX:HeapDumpPath=/dumps", # Dump location (mounted volume in docker-compose.override.yml)  
"-XX:+UseStringDeduplication", # G1 deduplicates String objects – saves 10-15% heap for string-heavy apps  
"-jar", "app.jar"]
```

10.5 Volume Strategy

Volume	Persists	What Is Lost on `docker-compose down -v`
`postgres-data`	Yes	All orders, products, customers — complete data loss
`redis-data`	Yes	Sessions, cache (quickly rebuilt), cart state
`mongo-data`	Yes	All product reviews
`cassandra-data`	Yes	All order activity audit log
`es-data`	Yes	Search index (can be rebuilt via `reindexAll()`)
`neo4j-data`	Yes	Purchase graph — recommendations lost until re-populated
`grafana-data`	Yes	Dashboard configurations
`jenkins-data`	Yes	Build history, job configurations
Kafka data	**No** (ephemeral)	All unprocessed Kafka messages — topic data lost

`docker-compose down` (**without** `-v`): Containers removed, volumes preserved — safe for development restarts.

`docker-compose down -v`: Containers AND volumes removed — full reset.

10.6 docker-compose.override.yml vs docker-compose.prod.yml

Setting	override.yml (dev)	prod.yml
Spring profile	`SPRING_PROFILES_ACTIVE=dev`	`SPRING_PROFILES_ACTIVE=prod`
Debug port	`5005:5005` exposed	Not exposed
TLS	Disabled	Enabled (Nginx with real cert)
Resource limits	None	`mem_limit: 2g` per service
Restart policy	`no`	`unless-stopped`
Source mounts	`./src:/app/src` (hot reload)	None
Log level	`DEBUG`	`INFO`

10.7 Monitoring Signals

Signal	Command	Alert
Container down	`docker ps --filter "status=exited"`	Any critical service exited
Memory pressure	`docker stats`	Container approaching `mem_limit`
Volume space	`docker system df`	> 80% disk used


```
[TRANSACTION COMMITS to PostgreSQL]

@TransactionalEventListener(AFTER_COMMIT):

9a. ProductSyncService.onProductCreated() [ASYNC]
    → productRepository.findById(553) [re-read from DB]
    → searchRepo.save(ProductDocument{stock:9})
    → Elasticsearch: updates products index stock field

9b. Neo4jSyncListener.onOrderCompleted() [ASYNC]
    → MERGE (c:Customer{id:1})-[:PURCHASED{orderId:155}]
      ->(p:Product{id:553})
    → Neo4j: PURCHASED relationship created
```

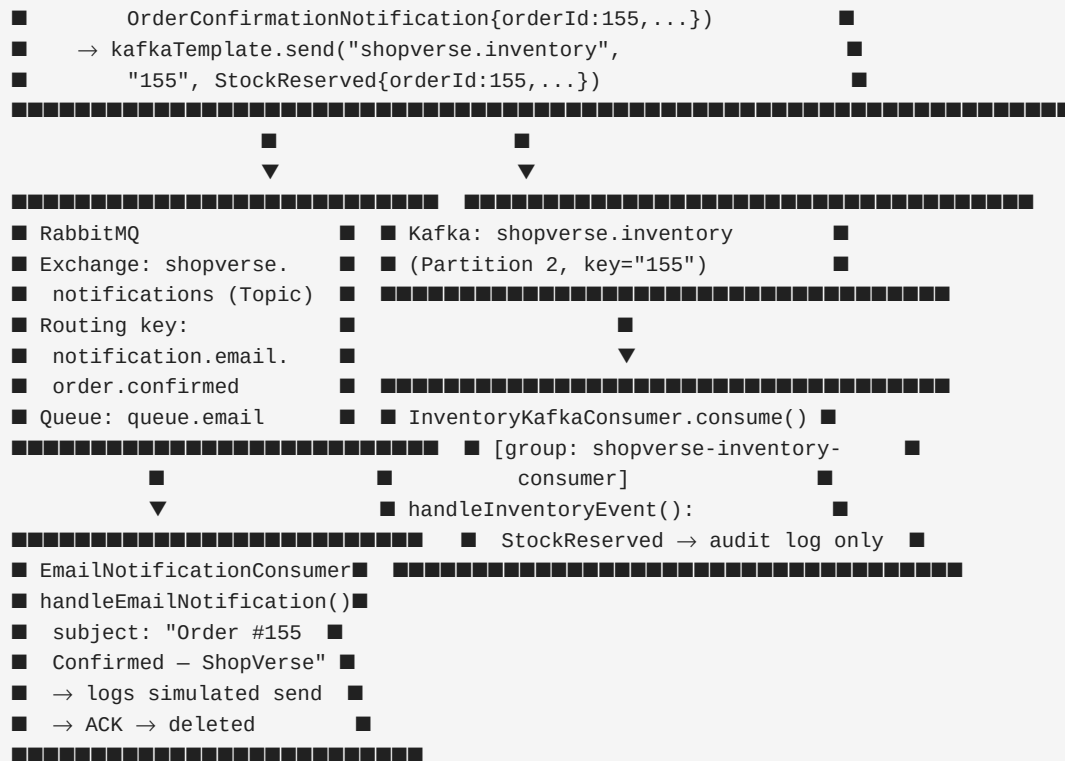
```

■ OrderKafkaProducer.onOrderEvent()
■ kafkaTemplate.send("shopverse.orders",
  "155", OrderPlaced{orderId:155,
  customerId:1, total:2249.10})

```

- Kafka: shopverse.orders (Partition 2, key="155")
- __TypeId__ header: OrderEvent\$OrderPlaced

```
■ OrderKafkaConsumer.consume() [group: shopverse-consumer]
■
■ handleOrderPlaced():
■   → notificationPublisher.publish(
```



Final Response to Browser:

HTTP 201 Created

{ "data": { "orderId": 155, "status": "CONFIRMED", "total": 2249.10 } }

Technologies touched in this single request:

✓ Nginx – reverse proxy

- ✓ Redis – JWT validation, rate limiting, idempotency key, cache hit
- ✓ PostgreSQL – order + order_items INSERT, products UPDATE, audit trigger
- ✓ Cassandra – order_activity INSERT (QUORUM)
- ✓ Kafka – shopverse.orders event published
- ✓ Elasticsearch – products index updated (async, post-commit)
- ✓ Neo4j – PURCHASED relationship created (async, post-commit)
- ✓ Kafka – shopverse.inventory event published (from order consumer)
- ✓ RabbitMQ – email notification published and consumed